# TaskFlow — Roteiro de Apresentação da Atividade 2

Lab de DAST, IaC e Observabilidade · Duração: ~10 min · Grupo Room 4 · gustavolucen4/DevSecOps-Room4

**Checklist Pré-Apresentação (5 min antes):**

1. Docker Compose rodando no host: docker compose up -d (App em :8080, Grafana em :3000)
2. Abas abertas: Home do Repo, PR #2 (Demo DAST), PR #3 (Demo IaC), Grafana Dashboard e Runbook NIST.
3. Terminal pronto na raiz para executar: ./scripts/simular-forca-bruta.sh http://127.0.0.1:8080 admin 25

## 0:00 - 1:30 Bloco 1: Contexto e Evolução — *Responsável: Gustavo Lucena*

**Tela / Onde Clicar:** Aba 1 (Home do repositório no GitHub)

**Objetivo:** Explicar a evolução da esteira estática (Atividade 1) para o ciclo completo em runtime (Atividade 2).

**Passo a Passo da Ação:**
• Mostrar a home do repositório oficial forkado e a branch main.
• Destacar que o repositório possui 9 Quality Gates ativos exigidos pelo ruleset da main.

**Fala Sugerida:** *"Olá a todos. Na Atividade 1 construímos nossa esteira focada em código estático. Na Atividade 2, fechamos o ciclo de DevSecOps: testamos a aplicação em execução com DAST no pipeline (OWASP ZAP), endurecemos a infraestrutura como código (Terraform e Docker Compose) auditada por Checkov e Trivy, e implementamos observabilidade defensiva com logs JSON correlacionados por request_id e runbook NIST SP 800-61."*

## 1:30 - 4:00 Bloco 2: Demonstração DAST — *Responsável: Filipe Almeida / Júlio Santos*

**Tela / Onde Clicar:** Aba 2 (PR #2: Regressão de XSS)

**Objetivo:** Comprovar que o DAST é um Quality Gate real que bloqueia o merge no GitHub Actions.

**Passo a Passo da Ação:**
1. Abrir o PR #2 que reintroduziu a falha trocando {{ error }} por {{ error | safe }}.
2. Apontar o check **DAST - ZAP Baseline** em vermelho e o botão de Merge **bloqueado**.
3. Explicar a espera ativa com curl no /login (evita o 'ZAP atacando o nada') e o ZAP Full Scan injetando o payload.
4. Mostrar a correção na main com autoescape limpo sem | safe em todos os sinks (q e erro).

**Fala Sugerida:** *"Para comprovar o gate em ação, criamos o PR #2 simulando uma regressão de XSS no login. O ZAP esperou a app subir, injetou o payload de ataque e reprovou o check na regra 40012 (A03 Injection). A proteção de branch bloqueou o merge. Na main, corrigimos todos os sinks com autoescape Jinja2 e removemos o debug."*

## 4:00 - 6:00 Bloco 3: Segurança em IaC — *Responsável: Frederico Cavalcanti / Pedro Holanda*

**Tela / Onde Clicar:** Aba 3 (PR #3: SSH Aberto) e docker-compose.yml

**Objetivo:** Demonstrar a auditoria automatizada de infraestrutura bloqueando configurações inseguras.

**Passo a Passo da Ação:**
1. Abrir o PR #3 com a porta 22 (SSH) aberta para 0.0.0.0/0 no Terraform.
2. Mostrar o Checkov (CKV_AWS_24) e o Trivy config reprovados, bloqueando o PR.
3. Apontar no código da main: SG restrito a allowed_cidr, S3 KMS com rotação e logging.
4. No Compose: 3 redes isoladas, SQLite em volume sem porta no host e Promtail sem docker.sock.

**Fala Sugerida:** *"Aqui no PR #3 mostramos o Quality Gate de IaC: ao abrir o SSH para 0.0.0.0/0, o Checkov e o Trivy travaram o merge. Na main corrigida, o SG só aceita nosso IP confiável, o S3 é privado e criptografado com KMS, e no Compose o banco e o Promtail são totalmente isolados sem expor portas."*

## 6:00 - 8:30 Bloco 4: Incidente ao Vivo & NIST — *Responsável: Gustavo Lucena / Filipe Almeida*

**Tela / Onde Clicar:** Terminal + Aba 4 (Grafana) + Aba 5 (Runbook NIST)

**Objetivo:** Exercitar o ataque simulado, mostrar o disparo no Grafana e percorrer as 4 fases do NIST.

**Passo a Passo da Ação:**
1. Executar no terminal: ./scripts/simular-forca-bruta.sh http://127.0.0.1:8080 admin 25.
2. No Grafana: mostrar o pico de falhas e a query de disparo: sum by (client_ip) (count_over_time({job='taskflow', event='login_failed'} | json [5m])) > 10.
3. No Runbook: percorrer Preparação, Detecção/Análise (query de sucesso vazia = não invadido), Contenção (rate limit Nginx) e Pós-Incidente.
4. Prova de ouro: busca por 'senha-errada' no Loki retorna 0 linhas (higiene de logs).

**Fala Sugerida:** *"Este é o momento mais importante da demo: estou disparando 25 tentativas de login com tokens CSRF reais. No Grafana, vejam o pico subindo. Nosso runbook NIST SP 800-61 entra em ação: checamos que não houve login_success, aplicamos a contenção no Nginx e comprovamos que nenhuma senha vazou no Loki."*

## 8:30 - 10:00 Bloco 5: Trade-offs & Conclusão — *Responsável: Todos os Integrantes*

**Tela / Onde Clicar:** Aba 1 (Rulesets da main e README)

**Objetivo:** Explicar decisões arquiteturais e responder às perguntas da banca.

**Passo a Passo da Ação:**
**1. Ausência de filtro paths: no iac-scan.yml (Prevenção de deadlock em PRs):**
• *Contexto:* O iac-scan.yml provê 3 checks obrigatórios (Required Status Checks) na proteção da main: IaC - Terraform validate, IaC - Checkov e IaC - Trivy config.
• *O Problema:* Se configurássemos um filtro como paths: ['infra/**', 'docker-compose*.yml'], um PR que alterasse apenas código de aplicação (ex: app.py ou templates/) **nunca dispararia o workflow**. O GitHub ficaria indefinidamente aguardando o status check ser reportado, criando um bloqueio perpétuo de merge.
• *Decisão:* O pipeline executa em todo PR. Dura ~1 min, elimina o risco de travar entregas e assegura auditoria contínua de IaC e containers.

**2. Política do OWASP ZAP (Flag -I e regras FAIL no .zap/rules.tsv):**
• *Contexto:* Em pipelines de CI, o ZAP roda localmente no runner em HTTP (127.0.0.1:5000), sem certificado TLS de produção.
• *O Problema:* No modo padrão, o ZAP reprovaria o job por avisos informativos de ambiente (ex: cookie sem flag Secure em HTTP ou falta de HSTS), gerando falsos-positivos e fadiga de alertas. Desligar o scanner ou usar continue-on-error, porém, desarmaria a proteção.
• *Decisão:* Execução com -I (ignorar warnings genéricos) somada ao arquivo rules.tsv versionado. Regras críticas de injeção (XSS 40012/40014, SQLi) e má configuração (CSP, anti-CSRF) são marcadas como **FAIL** (bloqueantes). Regras de infra/TLS no runner são documentadas como **WARN** com justificativa técnica formal.
• Finalizar e abrir para perguntas.

**Fala Sugerida:** *"Para concluir, destacamos dois trade-offs cruciais de engenharia: primeiro, o iac-scan.yml não possui filtro 'paths:' porque, sendo Required Check, impediria o merge de qualquer PR que não mexesse em infraestrutura. Segundo, no DAST utilizamos a flag '-I' com a lista explícita de FAIL no rules.tsv para focar em vulnerabilidades reais sem ruídos informativos de ambiente local. Estamos à disposição para perguntas!"*

## Perguntas Prováveis da Banca & Respostas Estratégicas

| Pergunta da Banca | Resposta Curta e Técnica |
|---|---|
| Por que o ZAP Baseline sozinho não pegava o XSS? | O Baseline é puramente passivo: só analisa respostas e headers. O XSS refletido só é comprovado quando injetamos payloads, por isso adicionamos o ZAP Full Scan ativo (10 min). |
| Por que não expuseram a porta do banco no host? | O SQLite opera em volume nomeado e roda exclusivamente na rede interna 'backend', sem mapeamento de porta para o host, eliminando qualquer superfície de ataque externa. |
| Por que desativaram a revisão de Code Owners no ruleset? | O repositório oficial é um fork (gustavolucen4) e o CODEOWNERS original apontava para @Shuralot. Manter a regra travaria o merge permanentemente por falta de permissão de um usuário externo. |
| Como garantem que senhas não chegam ao Loki? | Nosso logger JSON filtra ativamente e descarta chaves como password, token e csrf_token. Além disso, temos um teste automatizado (test_logs_are_json_without_secrets) que reprova o CI se senhas forem logadas. |